

GUIDE DU STAGIAIRE

DOCKER PRENDRE LA MER

Du premier conteneur à l'image optimisée. Vos consignes, concepts, commandes à exécuter, quiz et espaces de notes. Les corrigés sont présentés en séance.

6

MISSIONS

1

FIL ROUGE

10

COMMANDEMENTS



**SPARKS
FORMATION**

Guide stagiaire · Jour 1
Session inter · du 06/07/2026 au 08/07/2026
Docker Engine 29.x · Compose v2 · Swarm

Prendre la mer

Du premier conteneur à l'image optimisée

Au programme aujourd'hui

Aujourd'hui, vous partez de zéro : « qu'est-ce qui tourne vraiment quand je tape `docker run` ? ». Vous allez apprendre à piloter des conteneurs (Exercice 1), à diagnostiquer et réparer des pannes courantes (Exercice 2), puis fabriquer une image « à la main » pour en sentir les limites (Exercice 3), écrire un premier Dockerfile simple (Exercice 4), un Dockerfile applicatif complet pour le fil rouge ShipFast (Exercice 5), et enfin l'optimiser pour la production (Exercice 6). Chaque exercice produit quelque chose qui sert au suivant : gardez vos fichiers et vos images.

HORAIRE	ACTIVITÉ	OBJECTIF
9h00 - 9h35	Accueil, présentation du fil rouge ShipFast	Comprendre le fil conducteur des 3 jours
9h35 - 9h50	Quiz d'échauffement	Auto-positionnement
9h50 - 10h10	Théorie flash n°1 : VM vs conteneurs, architecture, écosystème, licences	Les bases conceptuelles
10h10 - 10h25	Pause	—
10h25 - 11h10	Exercice 1 — Les 10 commandements du conteneur	Un serveur web piloté de bout en bout
11h10 - 11h50	Exercice 2 — Le Détective : 5 conteneurs en détresse	Le réflexe de diagnostic
11h50 - 12h30	Exercice 3 — L'image à la main (commit, diff)... et pourquoi c'est mal	Votre première image, capturée en direct
13h30 - 13h45	Démo Docker Hub : tags & limites de pull	—
13h45 - 14h20	Exercice 4 — Premier Dockerfile : la carte de visite conteneurisée	Votre page web perso, en image versionnée
14h20 - 15h20	Exercice 5 — Conteneuriser ShipFast : le Dockerfile applicatif	L'API du fil rouge, en conteneur
15h20 - 15h35	Pause	—
15h35 - 16h30	Exercice 6 — Challenge « Opération régime »	La même API, nettement plus légère et non-root
16h30 - 16h45	Débrief : layers, cache, multi-stage, outillage (dive, hadolint)	—
16h45 - 17h00	Quiz de fin de journée	—

Votre environnement de travail

Vous travaillez sur une VM Linux dédiée, accessible depuis votre navigateur (terminal web) ou en SSH selon la fiche de connexion qui vous a été remise. Le dépôt `formation-docker/` est déjà cloné dans votre répertoire personnel.

Pour vérifier que votre environnement est prêt, exécutez :

```
docker version --format '{{.Server.Version}}'
ls ~/formation-docker
```

Si vous utilisez Rancher Desktop en local, vérifiez que le moteur actif est `dockerd` (moby) et non `containerd` (Preferences → Container Engine), et désactivez le Kubernetes intégré pour la semaine.

EN SECOURS

Si votre machine ne répond pas, utilisez [Play with Docker](#) dans votre navigateur : aucune installation requise, un environnement Docker complet est fourni en quelques secondes.

Les concepts du jour

VM vs conteneur

Une machine virtuelle virtualise le matériel : un hyperviseur, un système d'exploitation invité complet, des minutes de démarrage, des gigaoctets de RAM. Un conteneur virtualise le système d'exploitation : il s'appuie sur les namespaces et les cgroups du noyau Linux, démarre en millisecondes et consomme quelques mégaoctets.

Docker (2013) a démarré sur la technologie LXC puis l'a remplacée par sa propre pile (libcontainer, puis runc/containerd). L'apport majeur de Docker n'est pas l'isolation en elle-même (elle existait déjà) : c'est l'image portable, le Dockerfile et le registry — l'équivalent du conteneur maritime standardisé, appliqué au logiciel.

À RETENIR

Un conteneur ne contient pas de noyau : il partage celui de l'hôte. C'est la différence fondamentale avec une VM. Une image Docker n'est jamais modifiable : elle est immuable, on la reconstruit. Le conteneur ajoute seulement une couche d'écriture jetable par-dessus.

Sous le capot : de la commande au processus

Quand vous tapez une commande Docker, voici le chemin qu'elle parcourt : le client `docker` envoie une requête à l'API REST du démon `dockerd`, qui délègue à `containerd`, qui délègue lui-même à `runc` pour créer et lancer le processus conteneurisé. Vous pouvez observer cette chaîne avec :

```
docker system info
ps aux | grep -E "dockerd|containerd"
```

L'image : un empilement de couches

Une image Docker est un empilement de couches en lecture seule, chacune correspondant à une instruction du Dockerfile qui l'a construite. Ces couches sont partagées entre images quand elles sont identiques, ce qui économise de l'espace disque. Quand un conteneur démarre, Docker

ajoute par-dessus une fine couche d'écriture, propre à ce conteneur et perdue à sa destruction — d'où l'importance des volumes pour toute donnée à conserver (vu demain).

À RETENIR

Image = couches empilées, en lecture seule, immuables. Conteneur = une couche d'écriture temporaire posée sur une image. Détruire le conteneur ne touche jamais à l'image ; mais tout ce qui a été écrit dans cette couche d'écriture disparaît avec lui.

L'écosystème 2026

Quelques repères utiles pour situer les outils que vous manipulerez cette semaine :

- **Compose v2** : un plugin intégré au CLI, s'invoque avec `docker compose` (sans tiret).
- **BuildKit / buildx** : le moteur de build par défaut de Docker.
- **Docker Engine v29** : le containerd image store devient le stockage d'images par défaut, alignant Docker avec l'écosystème Kubernetes.
- **Docker Desktop et ses alternatives** : Rancher Desktop, Podman, Colima.
- **Docker Machine et Docker Cloud** sont abandonnés ; l'équivalent moderne est `docker context` et les offres cloud managées (aperçu au Jour 3).
- **Orchestration** : Kubernetes a gagné la place dominante (Mesos est retiré de l'Apache Foundation), Swarm reste en maintenance et convient très bien pour apprendre ou de petites productions.

Licences : ce qui est libre, ce qui ne l'est pas

- **Docker Engine** et toute la chaîne technique (CLI, Compose, BuildKit, containerd) : open source, licence Apache 2.0.
- **Docker Desktop** : licence commerciale au-delà de 250 salariés ou 10 millions de dollars de chiffre d'affaires — c'est l'installation qui compte, pas l'usage. Alternatives libres : Rancher Desktop, Podman, Colima.
- **Docker Hub** : 100 pulls / 6h par IP en anonyme, 200 en compte gratuit, illimité en offre payante — d'où l'intérêt de vous authentifier systématiquement et, en entreprise, de passer par un registry interne (mission dédiée demain matin).

À RETENIR

Docker Engine est libre (Apache 2.0). Docker Desktop peut être payant selon la taille de l'entreprise. Docker Hub limite le nombre de téléchargements anonymes par IP.

EXERCICE 1 · ~45 MIN

Les 10 commandements du conteneur

Objectif : piloter un serveur web réel de bout en bout — le lancer, y entrer, l'observer, l'arrêter, le détruire — pour comprendre concrètement qu'un conteneur, c'est un processus.

CE QUE VOUS DEVEZ FAIRE

Exécutez les 10 étapes ci-dessous dans l'ordre. Avant chaque commande accompagnée d'une question, essayez de deviner la réponse, puis vérifiez au clavier. Notez vos réponses : elles seront corrigées ensemble en séance.

```

# 1. La toute première fois
docker run hello-world
# Q1 : listez les 4 étapes que ce hello-world dit avoir traversées.

# 2. Un conteneur interactif
docker run -it ubuntu bash
cat /etc/os-release && ps aux          # → seulement 2 processus !
exit
# Q2 : pourquoi ps aux n'affiche que bash et ps ?

# 3. Où est passé mon conteneur ?
docker ps          # vide...
docker ps -a       # ...le voilà, "Exited"
# Q3 : pourquoi s'est-il arrêté quand vous avez tapé exit ?

# 4. Un vrai serveur, en arrière-plan
docker run -d --name web -p 8080:80 nginx:1.30-alpine
curl http://localhost:8080          # ou navigateur

# 5. Entrer dans un conteneur qui tourne
docker exec -it web sh
echo "<h1>Modifié en direct par $(hostname)</h1>" > /usr/share/nginx/html/index.html
exit
curl http://localhost:8080          # → votre page !
# Q4 : cette modification survivra-t-elle à un docker rm ?

# 6. Observer
docker logs -f web          # ouvrez un 2e terminal, faites 3 curl, regardez. Ctrl+C.
docker stats --no-stream # consommation réelle : quelques Mo !
docker inspect web --format '{{.NetworkSettings.IPAddress}}'

# 7. Le processus vu depuis l'hôte (Linux/WSL2)
docker top web
# Q5 : à quoi voit-on que le conteneur N'EST PAS une VM ?

# 8. Stop, start, la vie continue
docker stop web && docker ps -a
docker start web && curl localhost:8080          # la page modifiée est toujours là

# 9. La destruction (et la leçon)
docker rm -f web
docker run -d --name web -p 8080:80 nginx:1.30-alpine
curl localhost:8080          # page d'origine : la modification est-elle toujours là ?

# 10. Nettoyage de fin d'exercice
docker rm -f web && docker container prune -f

```

ATTENTION

Sous Windows, `curl` peut être un alias PowerShell qui ne se comporte pas comme le vrai `curl` : utilisez votre navigateur, ou `curl.exe`. Si le port 8080 est déjà pris sur votre machine, changez de port ou libérez-le avant de continuer.

Questions à préparer (réponses vues en séance) :

- Q1 : quelles sont les 4 étapes traversées par `docker run hello-world` ?

- Q2 : pourquoi `ps aux` n'affiche-t-il que `bash` et `ps` dans le conteneur ?
- Q3 : pourquoi le conteneur s'est-il arrêté quand vous avez tapé `exit` ?
- Q4 : votre modification a-t-elle survécu à la destruction du conteneur (étape 9) ? Pourquoi ?
- Q5 : à quoi voit-on, avec `docker top`, qu'un conteneur n'est pas une VM ?

DÉFI BONUS

Si vous avez terminé en avance, allez plus loin : ① Lancez `docker run -d --name boucle alpine sh -c 'while true; do date; sleep 1; done'`, puis observez `docker logs --tail 3 -t boucle` — à quoi sert l'option `-t` ? ② Trouvez le PID hôte du processus `nginx` avec `docker inspect -f '{{.State.Pid}}' web`, puis explorez `sudo ls -l /proc/<pid>/ns/` : que voyez-vous ? ③ Exécutez `docker exec web cat /sys/fs/cgroup/memory.max` : que représente cette valeur ? Ces trois manipulations montrent concrètement qu'un conteneur, c'est un processus + des namespaces + des cgroups.

NOTES

EXERCICE 2 · ~40 MIN

Le Détective — 5 conteneurs en détresse

Objectif : installer le réflexe de diagnostic `docker ps -a` → `docker logs` → `docker inspect`, la compétence qui distingue un utilisateur d'un pro.

CE QUE VOUS DEVEZ FAIRE

Vous êtes d'astreinte : 5 tickets viennent de tomber. Pour chaque ticket, reproduisez la panne avec la commande fournie, diagnostiquez avec les outils de l'Exercice 1 (`docker ps -a`, `docker logs`, `docker inspect`...), puis corrigez. Pour chaque ticket, notez la cause identifiée et la commande de correction.

```
# TICKET 1 — "Mon conteneur disparaît instantanément"
docker run -d --name t1 ubuntu

# TICKET 2 — "Impossible de recréer mon conteneur"
docker run -d --name t2 nginx:1.30-alpine
docker run -d --name t2 nginx:1.30-alpine

# TICKET 3 — "Le port refuse de se lier"
docker run -d --name t3a -p 8080:80 nginx:1.30-alpine
docker run -d --name t3b -p 8080:80 nginx:1.30-alpine

# TICKET 4 — "nginx tourne mais le site est injoignable"
docker run -d --name t4 nginx:1.30-alpine
curl http://localhost:8081

# TICKET 5 — "La base de données refuse de démarrer" (le plus réaliste)
```

```
docker run -d --name t5 postgres:18-alpine
docker ps      # ... t5 n'y est pas !
```

Questions à préparer (réponses vues en séance) :

- Pour chaque ticket : quelle est la cause exacte de la panne, et quelle commande la corrige ?
- Qu'est-ce que ces 5 pannes vous apprennent sur le comportement par défaut de Docker (option `-d`, ports, exposition réseau, configuration par variables d'environnement) ?

DÉFI BONUS

Deux pannes plus subtiles, pour les rapides : T6 — `docker run -d --name t6 --memory=6m nginx:1.30-alpine` : le conteneur redémarre en boucle. Diagnostiquez avec `docker inspect -f '{{.State.OOMKilled}}' t6` : que révèle ce champ ? T7 — un conteneur est « Up » mais `curl` renvoie « connection reset ». Diagnostiquez avec `docker inspect -f '{{.State.Health.Status}}' t7` : que révèle l'état de santé, et que peut-on en conclure sur le sens du statut « Up » ?

ASTUCE TECHNIQUE

Trois commandes à connaître pour un diagnostic rapide : `docker inspect -f '{{.State.ExitCode}}' X` donne le code de sortie exact (137 = tué/OOM, 139 = segfault, 143 = SIGTERM) ; `docker logs X 2>&1 | tail -50` capte aussi la sortie d'erreur (stderr) ; `docker inspect X | jq '.[0].State'` affiche l'état complet en un coup d'œil. Réflexe utile : lancer sans `-d` pour voir l'erreur directement à l'écran quand un conteneur refuse de démarrer.

NOTES

EXERCICE 3 · ~40 MIN

L'image à la main : commit, diff... et pourquoi c'est mal

Objectif : capturer votre toute première image personnalisée depuis un conteneur vivant — la méthode « artisanale » historique — pour comprendre, en la vivant, pourquoi le Dockerfile existe.

CE QUE VOUS DEVEZ FAIRE

À l'étape 9 de l'Exercice 1, vous avez perdu votre page modifiée en détruisant le conteneur. Cette fois, vous allez la capturer avant destruction, avec la méthode historique de Docker. Suivez les 5 étapes, puis répondez aux questions posées.

```
# 1. L'atelier : un nginx qu'on personnalise à chaud
docker run -d --name atelier -p 8080:80 nginx:1.30-alpine
docker exec -it atelier sh
  echo "<h1>Le site de $(hostname)</h1><p>Fait main, avec amour</p>" \
    > /usr/share/nginx/html/index.html
apk add --no-cache figlet      # on installe même un logiciel dans le conteneur
figlet "Docker !"
exit
```

```
# 2. L'audit : QU'EST-CE QUI a changé par rapport à l'image d'origine ?
docker diff atelier          # A = ajouté, C = modifié, D = supprimé
# Q1 : retrouvez votre index.html dans la liste. Que représente tout le reste ?

# 3. La capture : le conteneur devient une image
docker commit -m "page perso + figlet" atelier monsite:fait-main
docker images                # elle est là !

# 4. La preuve : on détruit tout, et on renaît de l'image
docker rm -f atelier
docker run -d --name renaissance -p 8080:80 monsite:fait-main
curl localhost:8080          # votre page a-t-elle survécu à la mort du conteneur ?
docker exec renaissance figlet "Immortel"

# 5. L'autopsie : que contient vraiment cette image ?
docker history monsite:fait-main
# Q2 : que dit la ligne du haut ? Peut-on savoir CE QUE contient cette couche ?
```

Questions à préparer (réponses vues en séance) :

- Q1 : que représente, dans `docker diff`, tout ce qui n'est pas votre `index.html` ?
- Q2 : que révèle (ou ne révèle pas) `docker history` sur le contenu réel de la couche créée par `commit` ?
- Cette image obtenue par `commit` est-elle reproductible ? versionnable dans Git ? auditable par une équipe sécurité ? automatisable en intégration continue ? Justifiez chaque réponse.

ASTUCE TECHNIQUE

`docker diff` est aussi un outil de sécurité simple : sur un conteneur de production, un diff inattendu (par exemple des binaires ajoutés dans `/tmp`) est un signal possible de compromission.

NOTES

Démo Docker Hub : tags et limites de pull

Cette démonstration vous présente l'anatomie d'une page d'image officielle sur Docker Hub : les tags, les variantes (`alpine`, `slim`...), la documentation des variables d'environnement, ainsi que la distinction entre Docker Official Image, Verified Publisher et image quelconque. Retenez la règle de base : épinglez toujours un tag précis (par exemple `postgres:18-alpine`) et n'utilisez jamais `latest` en production.

EXERCICE 4 · ~35 MIN

Premier Dockerfile : la carte de visite conteneurisée

Objectif : emballer votre page web personnelle dans une image versionnée avec le Dockerfile le plus simple possible, pour installer la boucle `write` → `build` → `run` → `itére`.

CE QUE VOUS DEVEZ FAIRE

Créez un dossier `moncv/`. Dedans, créez un fichier `index.html` à votre gloire (votre nom, vos 3 commandes Docker préférées de la matinée) et un `Dockerfile` de 2 lignes qui l'embarque dans une image `nginx`. Livrable : `curl localhost:8081` doit afficher votre page.

```
# moncv/index.html – exemple minimal (soyez créatifs)
<h1>Votre nom – vos 3 commandes préférées</h1>
<p>Mon top 3 : docker logs · docker exec -it · docker diff</p>

# moncv/Dockerfile
FROM nginx:1.30-alpine
COPY index.html /usr/share/nginx/html/

# La boucle complète
docker build -t moncv:v1 .
docker run -d --name cv -p 8081:80 moncv:v1
curl localhost:8081
```

Ensuite, entraînez-vous à la boucle d'itération : modifiez votre `index.html`, puis :

```
docker build -t moncv:v2 .          # → observez la vitesse : seule la couche COPY est
refaite
docker rm -f cv && docker run -d --name cv -p 8081:80 moncv:v2
docker images moncv                # v1 ET v2 coexistent : le versioning est gratuit
docker run -d --name cv-old -p 8082:80 moncv:v1  # les 2 versions tournent côte à côte !
```

BON À SAVOIR

Si le port de votre VM de TP est publié sur une adresse accessible publiquement, votre page sera réellement en ligne et consultable depuis un navigateur, à l'adresse indiquée sur votre fiche de connexion.

DÉFI BONUS

- ① Ajoutez `LABEL maintainer="vous@exemple.fr"` dans votre `Dockerfile` et retrouvez cette information avec `docker inspect`.
- ② Ajoutez une deuxième page `equipe.html` en utilisant `COPY . /usr/share/nginx/html/` à la place de votre `COPY` initial : qu'est-ce que cela change ?
- ③ Comparez `docker history moncv:v2` avec l'historique de l'image « fait-main » de l'Exercice 3 : qu'observez-vous de différent ?

Questions à préparer (réponses vues en séance) :

- Pourquoi le deuxième `docker build` (v2) est-il plus rapide que le premier ?
- Que change `COPY . /usr/share/nginx/html/` par rapport à un `COPY` ciblant un seul fichier ?
- En quoi `docker history` d'une image construite par `Dockerfile` diffère-t-il de celui d'une image obtenue par `docker commit` ?

NOTES

EXERCICE 5 · ~60 MIN

Conteneuriser ShipFast : le Dockerfile applicatif

Objectif : écrire l'image de l'API du fil rouge ShipFast — dépendances, code, port, commande de démarrage. Fichiers dans `cd ~/formation-docker/shipfast` (déjà cloné sur votre VM ; sinon clonez le dépôt de la formation).

Avant de commencer, explorez le code fourni : `server.js` (l'API : `POST /links` crée un lien court, `GET /:code` redirige, `GET /stats` compte les clics) et `package.json`. Sans les variables `DATABASE_URL` / `REDIS_URL`, l'API fonctionne en mode mémoire — c'est suffisant pour aujourd'hui ; la persistance arrive demain.

CE QUE VOUS DEVEZ FAIRE

Écrivez un Dockerfile qui : part d'une image Node.js 24, installe les dépendances (`npm install`), copie le code, expose le port 3000 et lance `node server.js`. BUILDEZ l'image sous le tag `shipfast:v1`, lancez-la, et prouvez que ça fonctionne avec les 3 tests ci-dessous. Aides autorisées : docs.docker.com, la page Docker Hub de l'image node.

```
# Tests de validation attendus :
docker build -t shipfast:v1 .
docker run -d --name ship -p 3000:3000 shipfast:v1
curl -X POST localhost:3000/links -H "Content-Type: application/json" \
  -d '{"url":"https://www.sparks-formation.com"}'
# → {"code":"Ab3dEf","short":"http://localhost:8080/Ab3dEf"}
curl -iL localhost:3000/Ab3dEf      # → 302 puis la page Sparks
curl localhost:3000/stats           # → {"links":1,"clicks":1,"store":"memory",...}

# 4e test, le plus satisfaisant : ouvrez http://vmX....:3000 dans votre navigateur
# → l'interface web ShipFast (servie par l'API) : créez un lien à la souris.
```

Pendant votre travail, testez aussi :

- Reconstituez après une modification de `server.js` et observez les couches marquées `CACHED`, puis après une modification de `package.json` : que se passe-t-il, et pourquoi ? (vous approfondirez cette question à l'Exercice 6)
- Différence `CMD` / `ENTRYPOINT` : essayez `docker run shipfast:v1 node --version` — que remplace cette commande ?
- `docker tag shipfast:v1 shipfast:latest` : un tag est une étiquette, pas une copie de l'image.
- `docker init` : cette commande génère automatiquement un Dockerfile et un fichier `Compose` de bonne qualité — un bon point de départ une fois que vous savez les lire.

ATTENTION

Un `COPY . .` sans fichier `.dockerignore` peut embarquer votre dossier `node_modules` local dans l'image (parfois incompatible avec Linux) et écraser celui installé pendant le build. Ce point sera corrigé à l'Exercice 6 avec un `.dockerignore`. Sur Apple Silicon / ARM, tout fonctionne, mais pensez à l'option `--platform linux/amd64` si l'image est destinée à un serveur x86.

Questions à préparer (réponses vues en séance) :

- Pourquoi une modification de `package.json` invalide-t-elle plus de couches en cache qu'une modification de `server.js` ?
- Quelle est la différence de comportement entre `CMD` et `ENTRYPOINT` ?

NOTES

EXERCICE 6 · ~55 MIN

Opération régime — l'image la plus légère gagne

Objectif : transformer votre `shipfast:v1` en une version de production : multi-stage, alpine, `.dockerignore`, utilisateur non-root, cache maîtrisé.

CE QUE VOUS DEVEZ FAIRE

Point de départ : votre `shipfast:v1`. Objectif : obtenir l'image la plus légère possible qui passe toujours les 3 tests de validation de l'Exercice 5, taguée `shipfast:v2`. Votre conteneur doit idéalement tourner en utilisateur non-root (vérifiable avec `docker exec ship whoami`).

Quatre pistes à explorer, dans l'ordre où elles sont le plus utiles :

- ① Regardez les variantes de l'image `node` disponibles sur Docker Hub.
- ② Que copie exactement `COPY . .` ? Créez le fichier qui permet de l'éviter.
- ③ L'ordre des instructions détermine le cache : copiez d'abord les fichiers de dépendances, le code ensuite.
- ④ A-t-on besoin de `npm` et de ses outils de build à l'exécution ? Renseignez-vous sur le « multi-stage build ».

```
# Une fois votre image obtenue, comparez-la aux outils suivants

# 1. dive – radiographie des couches (qu'est-ce qui pèse, où ?)
docker run --rm -it -v /var/run/docker.sock:/var/run/docker.sock \
  wgoodman/dive:latest shipfast:v1      # puis comparez avec :v2

# 2. hadolint – le linter de Dockerfile
docker run --rm -i hadolint/hadolint < Dockerfile

# 3. docker history – le juge de paix intégré
docker history shipfast:v2
```

À RETENIR

Chaque instruction du Dockerfile crée une couche en lecture seule, potentiellement partagée entre images (voir `docker system df`). Le cache de build s'invalide de haut en bas : le pattern « copier `package*.json` → `npm ci` → copier le reste du code » rend les reconstructions quasi instantanées quand seul le code change. Un build multi-stage permet d'obtenir une image finale sans les outils de build (compilateurs, gestionnaires de paquets), donc plus légère et avec une surface d'attaque réduite.

- Astuces utiles : `npm ci` est préférable à `npm install` dans une image (reproductible, plus rapide) ; en production, préférez des tags précis, idéalement épinglés par digest (`node:24-alpine@sha256:...`) ; `docker build --no-cache` force une reconstruction complète ; des images distroless ou Chainguard permettent de descendre encore en dessous de 100 Mo.
- Pour aller plus loin avec BuildKit : la ligne `# syntax=docker/dockerfile:1` en tête de Dockerfile débloque les cache mounts (`RUN --mount=type=cache,target=/root/.npm npm ci` , pour un cache de dépendances qui survit entre les builds) et les secrets de build (`RUN --mount=type=secret,id=npmmc ...` , pour utiliser un token privé au build sans qu'il finisse dans une couche de l'image).
- L'image `node` fournit un utilisateur `node` prêt à l'emploi : l'utiliser (avec l'instruction `USER node`) ne coûte qu'une ligne et ferme toute une classe d'attaques liées à l'exécution en root.

Questions à préparer (réponses vues en séance) :

- Quel est le facteur de réduction de taille obtenu entre votre `shipfast:v1` et votre `shipfast:v2` ?
- Quelles instructions précises de votre Dockerfile ont permis cette réduction ?
- Votre image finale tourne-t-elle en utilisateur non-root ? Comment l'avez-vous vérifié ?

DÉFI BONUS

Testez l'outil `docker-slim` (SlimToolkit), qui observe un conteneur en cours d'exécution et retire tout ce qui n'est pas utilisé, sans toucher au Dockerfile : `docker run --rm -v /var/run/docker.sock:/var/run/docker.sock dslim/slim build shipfast:v2`, puis comparez le résultat avec `docker images`.

NOTES

COMPLÉMENT IA

Un module dédié à l'IA appliquée à Docker sera abordé plus tard dans la formation (assistants intégrés, `docker init`). D'ici là, notez au fil des exercices les commandes ou astuces liées à l'IA que vous découvrez ou testez de votre côté : elles serviront de point de départ à ce module.

Quiz de fin de journée

#	QUESTION	VOTRE RÉPONSE
1	Que devient un conteneur quand son PID 1 meurt ?	
2	Commande pour voir les conteneurs arrêtés ?	
3	Premier réflexe quand un conteneur ne démarre pas ?	
4	Pourquoi <code>docker commit</code> est-il déconseillé pour fabriquer ses images ?	
5	À quoi sert <code>.dockerignore</code> ?	
6	Pourquoi copier <code>package.json</code> avant le reste du code ?	
7	Différence entre CMD et ENTRYPOINT ?	
8	Que gagne-t-on avec un build multi-stage ?	
9	Pourquoi éviter le tag <code>latest</code> en production ?	
10	Citez un outil d'analyse d'image ou de Dockerfile vu aujourd'hui.	